

SDV Home Lab

CAN-to-Cloud Software Defined Vehicle Prototype

This project demonstrates a simplified Software Defined Vehicle (SDV) architecture using Raspberry Pi, ESP32 controllers, and CAN communication.

The system simulates two vehicle functions:

- Autonomous Emergency Braking (AEB)
- Traffic Sign Memory (TSM)

A central compute node (Raspberry Pi) manages vehicle applications and distributes updates to edge ECUs over a CAN network.

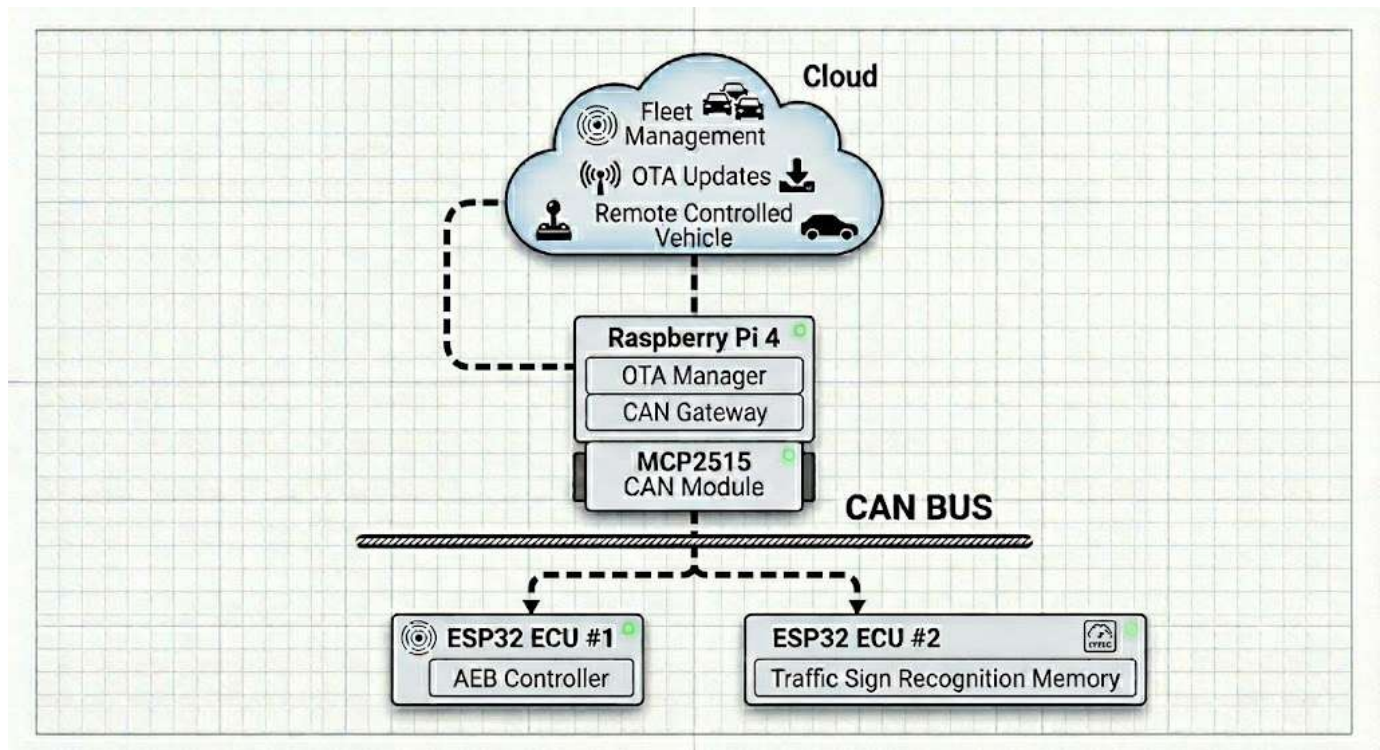
System Overview

The setup consists of:

Device	Purpose
Raspberry Pi 4	Central Compute Unit
ESP32 #1	AEB ECU
ESP32 #2	Traffic Sign Memory ECU
MCP2515 CAN Modules	CAN Interface Controllers
CAN Bus	Vehicle Communication Backbone

The architecture emulates a modern zonal vehicle network where application logic can be deployed independently to distributed ECUs.

Big picture(Architecture)



Hardware Requirements

Central Compute

- Raspberry Pi 4 (4GB or 8GB)
- 32GB SD Card
- Raspberry Pi OS

Edge ECUs

- ESP32 DevKit V1 (2 Units)

CAN Hardware

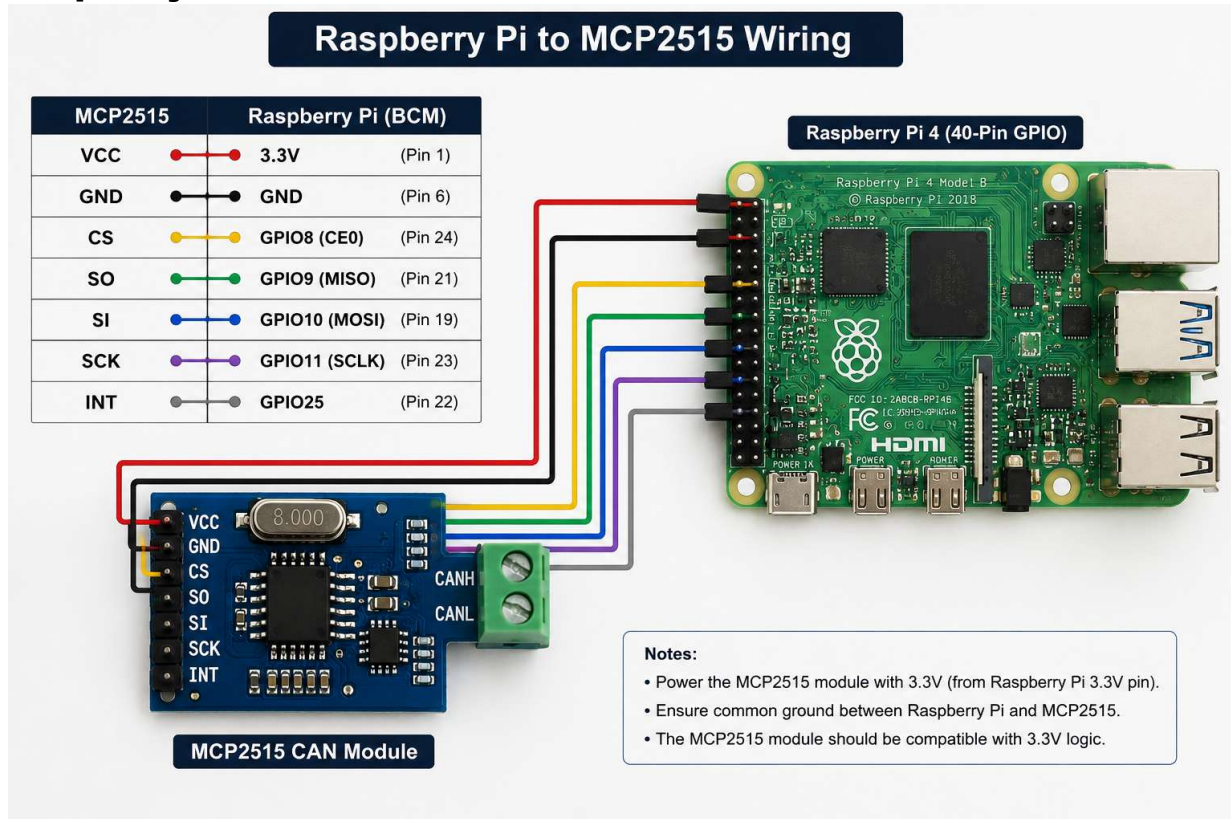
- MCP2515 CAN Module (3 Units)
- 120 Ohm Termination Resistors (2)

Power

- 5V USB Power Supply for Raspberry Pi
- USB Power for ESP32 Boards

Wirring Hereness Creations

Raspberry Pi to MCP2515



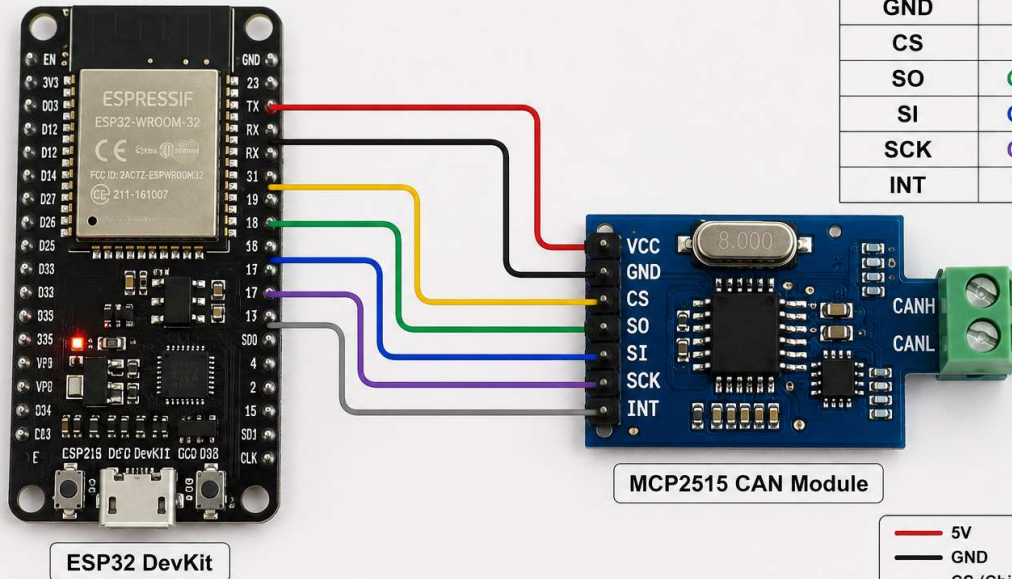
Raspberry Pi to MCP2515

MCP2515	Raspberry Pi (will ACT as TCU/GW/Central Compute)
VCC	3.3V
GND	GND
CS	GPIO8
SO	GPIO9
SI	GPIO10
SCK	GPIO11
INT	GPIO25

ESP32 to MCP2515

ESP32 ECU #1

ESP32 ECU #1 – MCP2515 Wiring



MCP2515	ESP32
VCC	5V
GND	GND
CS	GPIO5
SO	GPIO19
SI	GPIO23
SCK	GPIO18
INT	GPIO4

Note:

- Power the MCP2515 module with 5V (from ESP32 5V pin).
- Ensure common ground between ESP32 and MCP2515.

MCP2515 (CAN Module)	ESP32(EUC #1 - AEB controller)
VCC	5V
GND	GND
CS	GPIO5
SO	GPIO19
SI	GPIO23
SCK	GPIO18
INT	GPIO4

ESP32 ECU #2

MCP2515 (CAN Module)	ESP32(EUC #2 - Traffic Sign Memory)
VCC	5V
GND	GND
CS	GPIO5
SO	GPIO19
SI	GPIO23
SCK	GPIO18
INT	GPIO4

Follow the same picture as shown in ESP32 ECU #1

Important Points to Notes:

Make sure

- ✧ All MCP2515 modules are connected together.

Meaning

1	CANH of all three modules (MCP2515) Please connect together.
2	CANL of all three modules (MCP2515) Please connect together.

- ✧ 120 Ohm termination resistors are installed only at both ends of the bus.

Software Installation

Raspberry Pi

Assumptions!! Your Raspberry is up running and you are able to access your Pi4 with SSH. If not please refer my reference guide [./DAY-2/RaspberryPi4_initialSetup.pdf](#).

✧ Update System Packages

Always start with a full package index refresh and upgrade before installing hardware drivers. Stale package caches are a common source of dependency conflicts that are difficult to trace later.

```
sudo apt update
sudo apt upgrade -y
```

✧ Install CAN Utilities

The can-utils package provides the userspace tools you will use throughout this guide to monitor and inject CAN traffic. It is the de-facto standard toolchain for CAN on Linux.

```
sudo apt install can-utils -y
```

This installs four key binaries:

- candump** — capture and display all frames received on an interface
- cansend** — inject a single CAN frame with a specified ID and payload
- cangen** — generate random CAN traffic for stress-testing the bus
- canbusload** — measure bus utilisation as a percentage of bitrate capacity

✧ Enable the SPI Interface

SPI is the physical communication layer between the Raspberry Pi and the MCP2515 chip. It is disabled by default in Raspberry Pi OS to conserve GPIO resources and must be explicitly enabled through the raspi-config utility.

```
sudo raspi-config
```

Navigate through the menu hierarchy:

Interface Options → SPI → Enable → Finish

After exiting raspi-config, reboot to load the SPI kernel module:

```
sudo reboot
```

After reconnecting over SSH, verify that the SPI device nodes are present:

```
ls /dev/spi*
# Expected output:
/dev/spidev0.0 /dev/spidev0.1
```

✧ Configure the MCP2515 Driver Overlay

The Linux kernel learns about the MCP2515 hardware through a Device Tree overlay — a runtime hardware description file. Three lines in the Pi's boot configuration file are all that's needed to load the driver and bind it to the SPI bus.

```
sudo nano /boot/firmware/config.txt
```

Append the following three lines at the end of the file:

```
dtoverlay=spi-on
dtoverlay=mcp2515-can0,oscillator=16000000,interrupt=25
dtoverlay=spi-bcm2835
```

What each line does:

- ▶ `dtparam=spi=on` — redundant safety enable for SPI in the device tree (complements `raspi-config`).
- `dtoverlay=mcp2515-can0`** — loads the MCP2515 driver. The `oscillator=16000000` parameter must match the crystal frequency on your breakout board (typically 16 MHz). The `interrupt=25` maps the MCP2515 INT pin to GPIO 25 (physical pin 22).
- `dtoverlay=spi-bcm2835`** — ensures the BCM2835 SPI controller overlay is active, which is required for the MCP2515 overlay to bind correctly.

Save with `Ctrl+O` → `Enter` → `Ctrl+X`, then reboot:

```
sudo reboot
```

✧ Bring Up the CAN Interface

Once the Pi has booted with the overlay active, the `can0` interface will appear as a network device in the Linux kernel's network stack. Like an Ethernet interface, it must be brought up and configured with operational parameters before it can pass traffic.

```
# Bring up can0 at 500 kbit/s
sudo ip link set can0 up type can bitrate 500000

# Inspect interface details
ip -details link show can0
```

The bitrate (500 kbit/s) must match the bitrate configured on every other node on the bus — the two ESP32 ECUs in this setup. A bitrate mismatch is the single most common cause of a bus that looks electrically correct but produces no traffic.

✓ A healthy `can0` output from `ip -details` will show state `UP` along with the configured bitrate, `restart-ms` value, and zero error counts. Any non-zero error counts indicate a physical wiring problem.

✧ Flashing ESP32 ECUs

IDE and Library Setup

Download and install Arduino IDE 2.x from arduino.cc

Open Boards Manager and install the `esp32` package by Espressif Systems

Open Library Manager and install `MCP_CAN` by `coryjfowler` (and its dependency, `SPI` is built-in)

ECU #1 — Autonomous Emergency Braking (AEB) Controller

The AEB ECU reads simulated obstacle distance and vehicle speed sensors, evaluates whether braking is required based on a configurable distance threshold, and broadcasts its assessment over CAN every 100 ms.

```
# Open in Arduino IDE
firmware/aeb_ecu/aeb_ecu.ino

# Board selection
Tools → Board → ESP32 Dev Module

# Flash
Sketch → Upload
```

Once flashed, the AEB ECU begins transmitting CAN ID `0x100` frames immediately on power-up, without waiting for any handshake from the Raspberry Pi. This is consistent with how real ECUs behave — they broadcast their state continuously and let any listener on the bus consume the data.

5.3 ECU #2 — Traffic Sign Memory Controller

The Traffic Sign Memory ECU processes simulated camera input, classifies detected road signs, stores the most recent sign state in non-volatile memory, and publishes it over CAN at a regular interval. This pattern is common in ADAS stacks where sign state must persist across ignition cycles.

```
# Open in Arduino IDE
firmware/traffic_sign_ecu/traffic_sign_ecu.ino

# Flash
Sketch → Upload
```

After flashing, the ECU begins publishing CAN ID `0x200` frames containing the detected sign type, speed limit value, and a confidence score derived from the simulated classifier.

ECU #1 – AEB Controller

Open:

```
firmware/aeb_ecu/aeb_ecu.ino
```

Select board:

ESP32 Dev Module

Upload firmware.

Sketch -> Upload
The ECU starts transmitting AEB status messages over CAN.

ECU #2 – Traffic Sign Memory Controller

Open:

firmware/traffic_sign_ecu/traffic_sign_ecu.ino
Upload firmware.
The ECU starts publishing detected traffic signs and memory state.

CAN Message Definitions

CAN messages in this system follow a fixed-length 8-byte payload format. Each byte is assigned a specific meaning by the application-layer protocol — there is no built-in self-description in raw CAN frames. The definitions below serve as the data dictionary for this vehicle network.

AEB ECU — CAN ID 0x100

Byte	Field	Description
0	Obstacle Distance	Distance to nearest obstacle in metres (0–255 m range)
1	Brake Request	Boolean: 0x00 = no brake, 0x01 = brake requested
2	Vehicle Speed	Current speed in km/h (0–255 range)
3–7	Reserved	Set to 0x00; reserved for future expansion

Decode Example — AEB Frame

```
ID: 0x100
Data: 20 01 45 00 00 00 00 00
Byte 0 → 0x20 = 32 decimal → Obstacle Distance: 32 m
Byte 1 → 0x01 → Brake Request: TRUE (obstacle within threshold)
Byte 2 → 0x45 = 69 decimal → Vehicle Speed: 69 km/h
```

Traffic Sign Memory ECU — CAN ID 0x200

Byte	Field	Description
0	Sign Type	Enumeration: 0x00 = None, 0x01 = Speed Limit, 0x02 = Stop, 0x03 = Yield
1	Speed Limit	Limit value in km/h when Sign Type = 0x01; otherwise 0x00
2	Confidence	Classifier confidence as percentage (0–100)
3–7	Reserved	Set to 0x00; reserved for future expansion

Decode Example — Traffic Sign Frame

```
ID: 0x200
Data: 01 50 95 00 00 00 00 00
Byte 0 → 0x01 → Sign Type: Speed Limit
Byte 1 → 0x50 = 80 decimal → Speed Limit: 80 km/h
Byte 2 → 0x95 = 149... → Wait — 0x95 = 149 decimal
Confidence is stored as raw value (95 in this system = 95%)
```

✧ Monitoring CAN Traffic

With both ECUs transmitting and the Raspberry Pi's can0 interface up, you can observe the full vehicle network from the Pi's terminal. This is the equivalent of plugging a logic analyser into the CAN bus — every frame from every node is visible.

Capturing All Traffic

```
candump can0
```

Expected live output when both ECUs are active:

```
can0 100 [8] 20 01 45 00 00 00 00 00
can0 200 [8] 01 50 95 00 00 00 00 00
can0 100 [8] 1E 01 46 00 00 00 00 00
can0 200 [8] 01 50 96 00 00 00 00 00
...
```


Injecting a Test Frame

Use cansend to inject a synthetic AEB frame from the Raspberry Pi — useful for testing listener logic without needing the physical ECU connected:

```
cansend can0 100#2001450000000000
# Format: <CAN_ID>#<data_bytes_hex_no_spaces>
# This sends: ID=0x100, Obstacle=32m, Brake=TRUE, Speed=69 km/h
```

❖ Application Deployment Workflow

OTA update sequence · binary transfer · ECU activation

One of the defining characteristics of a Software Defined Vehicle is the ability to update ECU application logic remotely — without physically accessing the vehicle. In this setup, the Raspberry Pi acts as the deployment orchestrator, pushing new firmware binaries to each ESP32 ECU over the CAN bus.

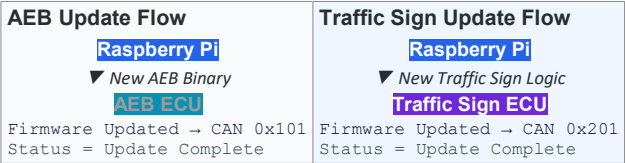
Deployment Sequence

The update flow follows a six-stage sequence regardless of which ECU is being updated:

- 1 Build Application
- 2 Generate Binary
- 3 Transfer Binary to ECU
- 4 Validate Firmware Version
- 5 Notify ECU to Activate
- 6 ECU Broadcasts Status

AEB and Traffic Sign Update Flows

Both ECUs follow the same pattern. The Raspberry Pi is the initiator; the ECU is the receiver. Completion is signalled by a dedicated CAN status frame on a separate ID (0x101 for AEB, 0x201 for Traffic Sign).



❖ Functional Demonstration

The following scenarios demonstrate how the system responds to simulated inputs — walking through the sensor input, the processing logic inside the ECU firmware, the output decision, and the resulting CAN frame that appears on the bus.

AEB Scenario — Emergency Brake Activation

Input	Obstacle Distance = 10 m, Vehicle Speed = 50 km/h
Threshold	AEB activates when obstacle distance < 15 m (configurable in firmware)
Processing	Distance (10 m) is below the 15 m threshold → Brake Request flag set to TRUE
CAN Output	ID: 0x100 Data: 0A 01 32 00 00 00 00 00
Decode	Byte 0 = 0x0A (10 m), Byte 1 = 0x01 (brake TRUE), Byte 2 = 0x32 (50 km/h)

Traffic Sign Memory Scenario — Speed Limit Detected

Input	Detected Sign = Speed Limit 60 km/h, Confidence = 91%
Processing	Sign type encoded as 0x01 (Speed Limit), limit stored as 0x3C (60), confidence as 0x5B (91)
Memory State	Previous sign overwritten; new state persisted in non-volatile memory
CAN Output	ID: 0x200 Data: 01 3C 5B 00 00 00 00 00
Decode	Byte 0 = 0x01 (Speed Limit), Byte 1 = 0x3C (60 km/h), Byte 2 = 0x5B (91%)